

Relatório de Atendimento Técnico

Cliente	Capricórnio
Projeto	Implantação de Stack de Observabilidade Zabbix/Grafana - Capricórnio

Consultor(es):	Samuel Gonçalves samuel.pereira@4linux.com.br Deborah Melo deborah.melo@4linux.com.br
Telefone:	11.2127-4700
E-mail:	consultoria@4linux.com.br
Apresentado em:	21 de maio de 2026

4LINUX SOFTWARE E COMÉRCIO DE PROGRAMAS LTDA
CNPJ: 04.491.152/0001-95
Telefone: (11) 2125-4747 Fax: (11) 2125-4777

Sumário

1 Objetivo **3**
1.1 Data de Início e Encerramento do Atendimento 3

2 Escopo do Atendimento **4**

3 Descrição dos Serviços Realizados **5**
3.1 Descrição Inicial do Ambiente 5
3.2 Atividades Realizadas 5
3.2.1 Preparação do Ambiente de Automação 5
3.2.2 Sequência Completa de Implantação 8
3.2.3 Referência Detalhada dos Playbooks 9
3.2.4 Mecanismos de Descoberta do Zabbix 32
3.2.5 Templates Utilizados 34
3.2.6 Macros Globais de Limiares 37
3.2.7 Scripts de Automação e Diagnóstico 38
3.3 Descrição do Ambiente Entregue 40

4 Problemas e Riscos Identificados **43**

5 Conclusões e Recomendações **44**

1 Objetivo

A Capricórnio solicitou a implantação de uma solução moderna de observabilidade para substituir o OpManager, que cobria parcialmente os três sites da empresa (BGP, SCL e SPO) sem visibilidade de bancos de dados, sistemas de energia ininterrupta nem infraestrutura wireless. O projeto envolveu a implantação de uma stack distribuída com Zabbix 7.0 LTS como plataforma de coleta e correlação, Grafana 13 para visualização executiva, PostgreSQL 16 em servidor dedicado como banco de dados, e três Zabbix Proxies distribuídos por site para coleta resiliente, totalizando seis servidores Ubuntu 24.04 LTS na stack central.

O engajamento abrangeu desde o hardening de segurança dos servidores até a configuração de 250+ dispositivos monitorados em cinco categorias: infraestrutura de rede (switches, firewalls, APs, NAS, UPS), servidores (Windows, Linux, Hyper-V, iDRAC), bancos de dados (SQL Server, PostgreSQL, MySQL, ClickHouse, Oracle), aplicações web e certificados SSL. Toda a configuração foi codificada em 26 playbooks Ansible, garantindo reprodutibilidade completa e facilitando a manutenção futura pela equipe de TI da Capricórnio. Este documento detalha cada playbook, cada mecanismo de descoberta, cada template e cada procedimento operacional necessário para que a equipe reproduza, expanda ou recupere qualquer parte da solução de forma autônoma.

Ao encerramento do atendimento, a stack encontra-se operacional nos três sites com monitoramento ativo de 250+ ativos, integração ao Active Directory via LDAPS para controle de acesso centralizado, nove dashboards Grafana por categoria de ativo e documentação técnica completa entregue ao cliente.

1.1 Data de Início e Encerramento do Atendimento

- Data de Início: 14/04/2026
- Data de Encerramento: 29/05/2026

2 Escopo do Atendimento

- **Em escopo:**
  - Instalação e configuração do Zabbix Server 7.0 LTS com PostgreSQL 16 dedicado
  - Implantação de três Zabbix Proxies em modo passivo (SPO, SCL, BGP)
  - Instalação e configuração do Grafana 13 com datasource Zabbix e plugin grafana-zabbix
  - Integração com Active Directory via LDAP/LDAPS para Zabbix e Grafana
  - Hardening dos seis servidores Linux Ubuntu 24.04 LTS da stack (SSH Ed25519, UFW, sysctl, unattended-upgrades)
  - Monitoramento de 32 switches Cisco CBS350/Catalyst 1200 via SNMPv3
  - Monitoramento de 5 firewalls FortiGate via API REST HTTPS
  - Monitoramento de 7 NAS QNAP via SNMPv3
  - Monitoramento de 27 UPS (24 Phoenixtec e 3 APC Smart-UPS) via SNMP
  - Monitoramento de 113 Access Points Aruba e 6 controllers via SNMPv3
  - Monitoramento de 31 servidores Windows Server 2022 e 4 workstations via Zabbix Agent 2
  - Monitoramento de 8 servidores Linux Ubuntu via Zabbix Agent 2
  - Monitoramento de 6 hosts Hyper-V via Zabbix Agent 2
  - Monitoramento de 7 servidores Dell PowerEdge via iDRAC SNMPv3
  - Monitoramento de 7 instâncias de banco de dados (3 SQL Server 2022, 1 PostgreSQL Veeam, 1 MySQL Lansweeper, 1 ClickHouse, 1 Oracle Systextil ERP)
  - Monitoramento de 22 endpoints web e 8 certificados SSL
  - Monitoramento de 5 Domain Controllers do Active Directory
  - Criação de 9 dashboards Grafana organizados por categoria de ativo
  - Automação completa via Ansible com 26 playbooks versionados em repositório Git
  - Elaboração de documentação técnica completa (manuais, inventário, guias de operação)
- **Fora de escopo:**
  - Configuração de alertas e notificações por e-mail ou canais de mensagens: não contratado neste engajamento; tratado como recomendação de próxima fase
  - Alta disponibilidade do PostgreSQL com replicação ou cluster Patroni: banco implantado em instância única conforme especificação inicial
  - Monitoramento de aplicações de negócio do ERP Systextil além da camada de banco Oracle
  - Criação de políticas de alertas, escalonamento e SLA no Zabbix: responsabilidade da equipe de TI da Capricórnio após entrega da plataforma

3 Descrição dos Serviços Realizados

3.1 Descrição Inicial do Ambiente

Ao início do atendimento, a Capricórnio operava com o OpManager como solução principal de monitoramento de infraestrutura. A ferramenta cobria parcialmente os três sites sem visibilidade unificada de bancos de dados, sistemas de energia ininterrupta nem infraestrutura wireless. Não havia dashboards executivos consolidados e o acesso ao sistema de monitoramento era feito por usuários locais, sem integração ao Active Directory.

Os seis servidores destinados à nova stack de observabilidade estavam provisionados com Ubuntu 24.04 LTS e acessíveis via SSH com o usuário `administrator`, porém sem hardening aplicado, sem UFW habilitado e sem configurações de segurança padronizadas. Não havia automação de configuração codificada nem inventário formal de ativos de rede em formato utilizável para importação direta no Zabbix.

A relação de ativos migrados do OpManager foi fornecida pelo cliente na planilha “relação opmanager.xlsx”, que serviu como base para o inventário inicial. Os switches Cisco de todos os sites já possuíam SNMPv3 configurado com perfil `authPriv` (SHA-256/AES-128), reduzindo o esforço de configuração inicial dos dispositivos de rede.

3.2 Atividades Realizadas

3.2.1 Preparação do Ambiente de Automação

3.2.1.1 Requisitos do Control Node

O control node é a máquina a partir da qual todos os playbooks Ansible são executados. Pode ser qualquer estação Linux/macOS ou WSL2 com acesso SSH aos servidores da stack. Os requisitos mínimos são:

```
# Versao minima do Ansible Core
ansible --version   # deve retornar 2.15 ou superior

# Python 3.10+ obrigatorio
python3 --version

# Instalar as collections necessarias (executar uma unica vez)
ansible-galaxy collection install community.zabbix grafana.grafana
    community.general
```

As collections instaladas cobrem os seguintes módulos utilizados pelos playbooks:

- `community.zabbix`: módulos `zabbix_server`, `zabbix_proxy`, `zabbix_agent`, `zabbix_host`, `zabbix_group`, `zabbix_macro`, `zabbix_discovery_rule`, `zabbix_action`, `zabbix_user_directory` e outros usados pelos playbooks de configuração via API
- `grafana.grafana`: módulo `grafana_datasource`, `grafana_dashboard` e instalação da ferramenta via role
- `community.general`: módulos utilitários como `ufw`, `sysctl`, `ini_file` e `blockinfile`

O acesso SSH aos servidores utiliza o usuário `administrator`. Antes do hardening, a autenticação é feita por senha configurada em `automacao/inventory/group_vars/all.yml`. Após o hardening, o playbook gera automaticamente um par de chaves Ed25519 em `automacao/files/ssh/` e deposita a chave pública em `~administrator/.ssh/authorized_keys` em todos os servidores. A partir desse ponto, toda autenticação é por chave.

3.2.1.2 Estrutura do Inventário Ansible

O inventário principal está em `automacao/inventory/hosts.yml` e define seis grupos de hosts correspondentes às funções de cada servidor:

```
zabbix_server:
  SPO-SRV-V01   (172.17.2.51)  Zabbix Server + Frontend Web

postgres:
  SPO-SRV-V02   (172.17.2.52)  PostgreSQL 16

grafana:
  SPO-SRV-V03   (172.17.2.53)  Grafana 13.0.1

zabbix_proxy:
  SPO-SRV-V04   (172.17.2.54)  Proxy SPO
  SCL-SRV-V08   (172.16.2.51)  Proxy SCL
  BGP-SRV-V08   (172.18.2.51)  Proxy BGP
```

O inventário de servidores Linux monitorados (fora da stack) está em `automacao/inventory/linux_servidores.yml`, organizado por grupos de site (linux_bgp, linux_scl, linux_spo). Cada grupo tem sua variável `zabbix_proxy_name` definida em `group_vars/` para que o agente aponte ao proxy correto do site.

3.2.1.3 Configuração das Variáveis Obrigatórias

Antes de executar qualquer playbook, preencher os campos `CHANGE_ME` nos arquivos abaixo. **Nunca commitar senhas em texto claro no repositório Git**; o uso de Ansible Vault é fortemente recomendado (ver seção de recomendações).

automacao/inventory/group_vars/all.yml

```
ansible_become_password: "CHANGE_ME"   # senha sudo do usuario
    administrator
zabbix_server_ip: "172.17.2.51"        # nao alterar
zabbix_version: "7.0"                  # nao alterar
```

automacao/inventory/group_vars/zabbix_server.yml

```
zabbix_server_dbpassword: "CHANGE_ME"  # senha do usuario zabbix no
    PostgreSQL
zabbix_web_dbpassword:    "CHANGE_ME"  # mesma senha (frontend usa o
    mesmo usuario)
```

automacao/inventory/group_vars/grafana.yml

```
grafana_ini:
  security:
    admin_password: "CHANGE_ME"        # senha do admin local do
        Grafana

grafana_datasources:
  - secureJsonData:
      password: "CHANGE_ME"            # senha do usuario Admin do
          Zabbix (API)

grafana_ldap_bind_password: "CHANGE_ME"  # senha da conta de servico
    LDAP
```

automacao/vars/db_plugins.yml

```
mssql_sessions:
  - password: "CHANGE_ME"    # SQL Server BGP-SRV-V04
  - password: "CHANGE_ME"    # SQL Server BGP-SRV-VAPP06
  - password: "CHANGE_ME"    # SQL Server SCL-SRV-VAPP02

pgsql_sessions:
  - password: "CHANGE_ME"    # PostgreSQL Veeam (BGP-SRV-VAPP02)
```

automacao/vars/switches.yml

Este arquivo contém as credenciais SNMPv3 (authpassphrase e privpassphrase) usadas em todos os switches Cisco e o inventário completo dos 32 switches com seus IPs de gerência. Preencher o campo `ip` de cada switch antes de executar `zabbix_switches.yml`. O script `gen_switch_vars.py` pode gerar este arquivo automaticamente a partir da planilha de inventário.

3.2.2 Sequência Completa de Implantação

A implantação deve ser executada na ordem abaixo. Cada etapa é um pré-requisito para as seguintes. O comando `site.yml` executa as etapas 2 a 5 de uma vez; para maior controle durante a primeira implantação, executar cada playbook individualmente.

Ordem	Playbook	Objetivo
1	`hardening.yml`	Hardening de segurança em todos os 6 servidores
2	`zabbix_server.yml`	Instala Zabbix Server + PostgreSQL 16 + Frontend Web
3	`grafana.yml`	Instala Grafana 13 + plugin Zabbix + datasource
4	`zabbix_proxy.yml`	Instala e configura os 3 proxies Zabbix
5	`zabbix_agents.yml`	Instala o Agent 2 nos 6 servidores da stack
6	`configure_etc_hosts.yml`	Distribui entradas /etc/hosts internas
7	`zabbix_register_proxies.yml`	Registra os 3 proxies via API Zabbix
8	`zabbix_infra_setup.yml`	Cria grupos, macros, regras de discovery e actions
9	`zabbix_ldap.yml`	Configura LDAP/AD no Zabbix
10	`grafana_ldap.yml`	Configura LDAP/AD no Grafana
11	`grafana_dashboards.yml`	Importa os 9 dashboards Grafana
12	`zabbix_agente_linux.yml`	Instala Agent 2 nos servidores Linux monitorados
13	`configure_db_plugins.yml`	Configura plugins de banco de dados nos proxies
14	`zabbix_switches.yml`	Cadastra os 32 switches Cisco via API
15	`zabbix_criar_ups.yml`	Cadastra os UPS via API

Ordem	Playbook	Objetivo
16	`zabbix_databases.yml`	Cadastra e configura monitoramento de bancos
17	Configuração via UI Zabbix	Configura Media Types Telegram e Teams + Actions de alerta
18	`zabbix_web_monitor.yml`	Cria cenários web e verificações de certificados SSL
19	`zabbix_ad.yml`	Cadastra os 5 Domain Controllers para monitoramento
20	`configure_proxy_monitoring.yml`	Configura self-monitoring dos proxies

Todos os comandos abaixo devem ser executados a partir do diretório `automacao/`:

```
cd automacao/
```

3.2.3 Referência Detalhada dos Playbooks

3.2.3.1 Fase 1: Hardening e Segurança dos Servidores

Playbook `hardening.yml`

Aplica a role `hardening` em todos os seis servidores da stack. Executa quatro tarefas independentes em sequência: SSH, firewall, sysctl e atualizações automáticas. Ao final, verifica os resultados de cada medida aplicada.

```
ansible-playbook -i inventory/hosts.yml playbooks/hardening.yml
```

Para aplicar em um único servidor (por exemplo, após reinstalação):

```
ansible-playbook -i inventory/hosts.yml playbooks/hardening.yml --limit
    =SPO-SRV-V01
```

Tarefa SSH (`roles/hardening/tasks/ssh.yml`):

O Ansible gera um par de chaves Ed25519 no control node e salva em `files/ssh/capricornio_ed25519` (privada) e `capricornio_ed25519.pub` (pública). A chave pública é depositada em ~

`administrator/.ssh/authorized_keys` com permissão `0600`. Em seguida, o `sshd_config` recebe as configurações de hardening abaixo, aplicadas via template Jinja2:

Diretiva sshd_config	Valor configurado	Motivo
Protocol	2	Proíbe SSHv1, que tem vulnerabilidades conhecidas
Port	22	Porta padrão; restrita via UFW
PermitRootLogin	no	Impede acesso direto como root
MaxAuthTries	3	Limita tentativas de força bruta
MaxSessions	10	Previne uso excessivo de multiplexação
PubkeyAuthentication	yes	Habilita autenticação por chave
PasswordAuthentication	yes	Mantido sim durante provisionamento inicial
X11Forwarding	no	Previne tunelamento gráfico desnecessário
AllowAgentForwarding	no	Reduz superfície de ataque de encaminhamento de agente
AllowTcpForwarding	local	Permite apenas tunelamento local (necessário para Ansible)
LoginGraceTime	30s	Fecha conexões de autenticação lentas
ClientAliveInterval	300s	Detecta conexões inativas a cada 5 minutos
ClientAliveCountMax	3	Encerra sessão após 3 intervalos sem resposta (15 min)
LogLevel	VERBOSE	Registra detalhes de autenticação para auditoria

Tarefa Firewall (`roles/hardening/tasks/firewall.yml`):

Instala e configura o UFW com política padrão `deny incoming / allow outgoing`. As regras por grupo são definidas em `group_vars/` de cada grupo:

Grupo	Portas liberadas (TCP)	Origem
zabbix_server	22 (SSH), 80 (HTTP), 443 (HTTPS), 10051 (Trapper)	qualquer
postgres	22 (SSH), 5432 (PostgreSQL)	apenas 172.17.2.51
grafana	22 (SSH), 3000 (Grafana UI)	qualquer
zabbix_proxy	22 (SSH), 10051 (Trapper)	qualquer

Tarefa Sysctl (roles/hardening/tasks/sysctl.yml):

Aplica 25 parâmetros de kernel via /etc/sysctl.d/99-hardening.conf. Os parâmetros mais relevantes:

Parâmetro	Valor	Efeito
net.ipv4.tcp_syncookies	1	Proteção contra ataques SYN flood
net.ipv4.conf.all.rp_filter	1	Verifica origem de pacotes (anti-spoofing)
net.ipv4.conf.all.accept_redirects	0	Rejeita redirecionamentos ICMP
net.ipv4.conf.all.send_redirects	0	Não envia redirecionamentos ICMP
net.ipv4.conf.all.accept_source_route	0	Rejeita roteamento de fonte
net.ipv4.icmp_echo_ignore_broadcasts	1	Proteção contra smurf attacks
net.ipv4.tcp_fin_timeout	30	Acelera fechamento de conexões TIME_WAIT
net.ipv4.tcp_keepalive_time	1200	Keepalive a cada 20 minutos
net.core.somaxconn	65535	Fila máxima de conexões TCP
net.ipv4.tcp_max_syn_backlog	4096	Backlog de SYN para proteção contra flood

Parâmetro	Valor	Efeito
`vm.swappiness`	10	Minimiza uso de swap (preserva I/O para PostgreSQL)
`kernel.randomize_va_space`	2	ASLR completo (anti-exploits de memória)
`kernel.kptr_restrict`	2	Oculta ponteiros do kernel em /proc
`kernel.dmesg_restrict`	1	Restringe acesso ao log do kernel
`fs.protected_symlinks`	1	Previne ataques de symlink em diretórios tmp
`fs.protected_hardlinks`	1	Previne ataques de hardlink

Tarefa Atualizações Automáticas (`roles/hardening/tasks/unattended_upgrades.yml`):

Instala e configura `unattended-upgrades` para aplicação automática de patches de segurança:

- Aplica apenas atualizações da origin `Ubuntu:24.04` com label `Ubuntu` e `security` no suite name
- Reinicialização automática desabilitada: patches de segurança são aplicados sem reiniciar automaticamente; reinicializações devem ser agendadas manualmente
- Remoção automática de dependências obsoletas e kernels antigos habilitada
- Verificação diária configurada via APT periodic com download e aplicação em ciclos de 24 horas

Verificação pós-hardening:

O próprio playbook `hardening.yml` executa verificações ao final via `assert`:

```
# Verificar que PasswordAuthentication esta configurada
grep -i PasswordAuthentication /etc/ssh/sshd_config

# Verificar UFW ativo
ufw status

# Verificar sysctl aplicado
sysctl net.ipv4.tcp_syncookies

# Verificar unattended-upgrades rodando
systemctl is-active unattended-upgrades
```

3.2.3.2 Fase 2: Instalação da Stack Central

Playbook `zabbix_server.yml`

Instala o Zabbix Server 7.0 LTS e o frontend web com Nginx + PHP 8.3 FPM no nó SPO-SRV-V01. Na mesma execução, instala e configura o PostgreSQL 16 no nó SPO-SRV-V02.

```
ansible-playbook -i inventory/hosts.yml playbooks/zabbix_server.yml
```

O playbook utiliza as roles `community.zabbix.zabbix_server` e `community.zabbix.zabbix_web`. As principais configurações aplicadas (definidas em `group_vars/zabbix_server.yml`):

- **Banco de dados**: tipo `pgsql`, host `172.17.2.52`, porta `5432`, banco `zabbix`, usuário `zabbix`
- **Web server**: Nginx com PHP 8.3 FPM; remove o vhost padrão do Nginx que conflitaria com o vhost do Zabbix
- **API Server Name**: “Zabbix Capricornio” (identificação do servidor na interface web)
- **UFW**: abre as portas 22, 80, 443 e 10051 no servidor

Após a instalação, o frontend web fica disponível em `http://172.17.2.51`. Login inicial: usuário `Admin`, senha `zabbix` (alterar imediatamente em Administration > Users).

O script `automacao/scripts/zabbix_server_tune.sh` ajusta os parâmetros de performance após a instalação. Ele cria um backup do `zabbix_server.conf` com data e hora antes de qualquer alteração:

```
# Parametros ajustados pelo script
ValueCacheSize=64M    # cache de valores para calculos de triggers
StartPingers=3        # threads paralelas para checagens ICMP
```

```
# Executar tuning de performance (como root no SPO-SRV-V01)
bash scripts/zabbix_server_tune.sh
```

Playbook `grafana.yml`

Instala o Grafana 13.0.1 no nó SPO-SRV-V03 e configura o datasource Zabbix.

```
ansible-playbook -i inventory/hosts.yml playbooks/grafana.yml
```

Após a instalação da role `grafana.grafana`, o playbook executa as seguintes tarefas adicionais:

1. Instala o plugin `alexanderzobnin-zabbix-app` via `grafana-cli`, especificando `--homepath /usr/share/grafana` e `--pluginsDir /var/lib/grafana/plugins`
2. Aguarda a porta 3000 ficar disponível (timeout 60 segundos)
3. Define a senha do admin via CLI (`grafana-cli admin reset-admin-password`)

4. Habilita o plugin Zabbix via chamada REST à API do Grafana (`/api/plugins/alexanderzobnin-zabbix-app/settings`)

O datasource Zabbix é configurado apontando para `http://172.17.2.51/api_jsonrpc.php` com o usuário `Admin` do Zabbix. A URL do Grafana é `http://172.17.2.53:3000`. Login inicial: usuário `admin`, senha definida em `group_vars/grafana.yml`.

Playbook `zabbix_proxy.yml`

Instala o Zabbix Proxy 7.0 nos três nós de proxy (SPO-SRV-V04, SCL-SRV-V08, BGP-SRV-V08).

```
ansible-playbook -i inventory/hosts.yml playbooks/zabbix_proxy.yml
```

Configurações definidas em `group_vars/zabbix_proxy.yml`:

- **Modo de operação**: passivo (modo 0). No modo passivo, o Zabbix Server inicia a conexão com o proxy para buscar dados; o proxy não precisa ter conectividade de saída para o servidor, apenas a porta 10051/TCP precisa estar acessível do servidor para o proxy
- **Banco de dados local**: SQLite3 (`/var/lib/zabbix/zabbix_proxy.db`), sem dependência de servidor de banco externo
- **Porta de escuta**: 10051/TCP
- **Servidor Zabbix**: 172.17.2.51:10051
- **Diretório de dados**: `/var/lib/zabbix/` com ownership `zabbix:zabbix`

Playbook `zabbix_agents.yml`

Instala o Zabbix Agent 2 nos seis servidores da própria stack (Zabbix Server, PostgreSQL, Grafana e os três proxies), permitindo que a própria infraestrutura de monitoramento seja monitorada.

```
ansible-playbook -i inventory/hosts.yml playbooks/zabbix_agents.yml
```

A configuração do agente é gerada pelo template `automacao/templates/zabbix_agent2.conf.j2`:

```
PidFile=/run/zabbix/zabbix_agent2.pid
LogFile=/var/log/zabbix/zabbix_agent2.log
LogFileSize=10
Server=127.0.0.1,{{ zabbix_server_ip }}
ServerActive={{ zabbix_server_ip }}
Hostname={{ inventory_hostname }}
Timeout=10
Include=/etc/zabbix/zabbix_agent2.d/*.conf
```

Para os proxies remotos (SCL e BGP), o `ServerActive` aponta para o proxy local (o próprio hostname), e o `Server` inclui o IP do proxy e o IP do servidor. Isso permite que o proxy monitore a si mesmo e que o Zabbix Server também colete dados diretamente quando necessário.



Playbook `configure_etc_hosts.yml`

Distribui entradas de resolução de nomes internos no `/etc/hosts` de todos os servidores da stack. As entradas são gerenciadas por um bloco Ansible para evitar conflitos com entradas manuais.

```
1 ansible-playbook -i inventory/hosts.yml playbooks/configure_etc_hosts.
    yml
```

As entradas configuradas (definidas em `automacao/vars/etc_hosts_internal.yml`) resolvem hostnames do domínio `.capricornio.corp.br` cujos registros DNS não são alcançáveis a partir dos servidores da stack sem VPN:

- `lansweeper.capricornio.corp.br` (172.18.2.49)
- `nodered.capricornio.corp.br` (172.16.100.250) com três instâncias nas portas 1880, 1881 e 1882
- `cisco.capricornio.corp.br` (172.16.2.144)

Essas entradas são necessárias para que o Zabbix Agent colete métricas de URLs que usam esses hostnames nos cenários web configurados em `zabbix_web_monitor.yml`.

3.2.3.3 Fase 3: Configuração da Estrutura do Zabbix

Playbook `zabbix_register_proxies.yml`

Registra os três proxies no Zabbix Server via API. Este playbook deve ser executado antes de `zabbix_infra_setup.yml`, pois as regras de discovery precisam dos IDs dos proxies para serem criadas.

```
1 ansible-playbook -i inventory/hosts.yml playbooks/
    zabbix_register_proxies.yml
```

O playbook usa o módulo `community.zabbix.zabbix_proxy` para criar (ou atualizar, se já existirem) os três registros de proxy no Zabbix com modo passivo (proxy_mode=0). Após a execução, verificar em Administration > Proxies que os três proxies aparecem com status “Active” e última comunicação recente.

Playbook `zabbix_infra_setup.yml`

Este é o playbook mais importante da implantação. Ele configura toda a estrutura lógica do Zabbix: grupos de hosts, macros globais, regras de descoberta de rede e ações automáticas de classificação. Deve ser executado após `zabbix_register_proxies.yml`.

```
1 ansible-playbook -i inventory/hosts.yml playbooks/zabbix_infra_setup.
    yml
```

1. Criação de Grupos de Hosts (36 grupos)

O playbook cria 36 grupos organizados hierarquicamente por categoria e site. A hierarquia usa / como separador, permitindo herança de permissões no Zabbix:

Grupos criados	Descrição
`Switches Cisco`/`BGP`, /`SCL`, /`SPO`, /`DCT`, /`FIA`	Switches Cisco por site
`Firewalls`/`BGP`, /`SCL`, /`SPO`, /`DCT`, /`FIA`	Firewalls por site
`NAS QNAP`/`BGP`, /`SCL`, /`SPO`	NAS QNAP por site
`UPS`/`BGP`, /`SCL`, /`SPO`	Sistemas de no-break por site
`Links WAN`/`BGP`, /`SCL`, /`SPO`	Links de WAN por site
`Access Points Aruba`/`BGP`, /`SCL`, /`SPO`	APs Aruba por site
`Servidores Windows`/`BGP`, /`SCL`, /`SPO`	Servidores Windows por site
`Servidores Linux`	Servidores Linux (todos os sites)
`Hyper-V`/`BGP`, /`SCL`	Hosts Hyper-V por site
`iDRAC Dell`/`BGP`, /`SCL`, /`SPO`	iDRAC Dell por site
`Bancos de Dados`	Instâncias de banco de dados
“Aplicações e URLs”	Cenários web e aplicações
`Certificados SSL`	Verificações de certificado
`Equipamentos Especiais`	Equipamentos fora das categorias acima
“Em Classificação”	Hosts descobertos aguardando classificação

Os grupos DCT e FIA foram criados preventivamente para acomodar sites futuros sem necessidade de nova execução do playbook.

2. Configuração de Macros Globais (16 macros)

As macros globais definem valores padrão reutilizados por todos os templates. Configuradas em Administration > Macros no Zabbix, elas evitam que cada host precise ter suas próprias definições de credenciais e limites:

Macro	Valor configurado	Uso
`{$SNMP.USERNAME}`	zabbix_snmp	Usuário SNMPv3 global

Macro	Valor configurado	Uso
{$SNMP.AUTH.PASSPHRASE}	(ver vars/switches.yml)	Senha de autenticação SNMPv3
{$SNMP.PRIV.PASSPHRASE}	(ver vars/switches.yml)	Senha de privacidade SNMPv3
{$SNMP.COMMUNITY}	(ver vars/switches.yml)	Community SNMPv2c (UPS)
{$SNMP.COMMUNITY.CISCO.LEGACY}	(ver vars/switches.yml)	Community legada para switches Cisco mais antigos
{$SNMP.COMMUNITY.WAGO}	(ver vars/switches.yml)	Community para CLPs WAGO
{$CPU.UTIL.CRIT}	90	Percentual de CPU para trigger crítica
{$MEMORY.UTIL.MAX}	90	Percentual de memória para trigger de aviso
{$IF.UTIL.MAX}	80	Percentual de utilização de interface de rede
{$DISK.PFREE.MIN.WARN}	20	Percentual mínimo livre em disco (aviso)
{$DISK.PFREE.MIN.CRIT}	10	Percentual mínimo livre em disco (crítico)
{$UPS.BATTERY.CAPACITY.MIN.WARN}	40	Capacidade mínima de bateria de UPS (%)
{$CERT.EXPIRY.WARN}	30	Dias antes do vencimento para alerta de certificado SSL
{$ALERT.EMAIL}	infraestrutura.n2@capricornio.com.br	E-mail de destino para notificações

3. Regras de Descoberta de Rede

Três regras de network discovery são criadas, uma por site, cada uma executada pelo proxy do respectivo site. O discovery varre sub-redes de gerência (VLAN de gerência é a rede .23. de cada site):

Regra	Proxy	Sub-rede varrida	Verificações
Discovery BGP	BGP-SRV-V08	172.18.23.0/24	ICMP ping + SNMPv3 sysDescr

Regra	Proxy	Sub-rede varrida	Verificações
Discovery SCL	SCL-SRV-V08	172.16.23.0/24	ICMP ping + SNMPv3 sysDescr
Discovery SPO	SPO-SRV-V04	172.17.23.0/24	ICMP ping + SNMPv3 sysDescr

Cada regra executa duas verificações por IP:

1. **ICMP ping**: verifica se o IP está respondendo (tipo `ICMP`)
2. **SNMPv3 sysDescr**: lê o OID `.1.3.6.1.2.1.1.1.0` (system.sysDescr), que retorna a string de identificação do dispositivo (ex: "Cisco CBS350-48P", "FortiGate-60F", "QNAP TS-453D")

O intervalo de execução de cada regra é de **3600 segundos (1 hora)**. Para forçar uma descoberta imediata sem aguardar o intervalo, usar o playbook `zabbix_reset_discovery.yml` (descrito na seção de manutenção).

4. Actions de Classificação Automática por Discovery

Quando a regra de discovery encontra um dispositivo, uma Action verifica o conteúdo do `sysDescr` retornado e classifica o host automaticamente. A lógica de classificação por `sysDescr`:

Condição (sysDescr contém)	Grupo destino	Template aplicado
"CBS350" ou "Catalyst"	`Switches Cisco/SITE`	Net Cisco IOS SNMP
"FortiGate"	`Firewalls/SITE`	FortiGate by HTTP
"QNAP"	`NAS QNAP/SITE`	QNAP NAS by SNMP
"APC"	`UPS/SITE`	APC UPS by SNMP
"Aruba"	`Access Points Aruba/SITE`	Aruba IAP by SNMPv3
"iDRAC" ou "Dell PowerEdge"	`iDRAC Dell/SITE`	Dell iDRAC by SNMP
(qualquer outro)	"Em Classificação"	(nenhum)

O "SITE" é determinado pelo prefixo do IP: 172.16.x.x = SCL, 172.17.x.x = SPO, 172.18.x.x = BGP.

Dispositivos que caem no grupo "Em Classificação" devem ser revisados manualmente ou pelo playbook `zabbix_classificar_hosts.yml`.

5. Actions de Registro Automático de Agentes (Auto-registration)

Quando um Zabbix Agent 2 inicia pela primeira vez e se registra no servidor, uma Action processa o registro com base no campo `HostMetadata` enviado pelo agente. O template `zabbix_agent2.conf.j2` configura o `HostMetadata` automaticamente durante a instalação:

HostMetadata configurado	Grupo destino	Template aplicado
`windows-server`	`Servidores Windows/SITE`	Windows OS by Zabbix agent active
`linux-server`	`Servidores Linux`	Linux by Zabbix agent 2 active
`hyperv`	`Hyper-V/SITE`	Hyper-V by Zabbix agent 2

O SITE é extraído dos dois primeiros caracteres do hostname: prefixo “BGP” manda para BGP, “SCL” para SCL, “SPO” para SPO.

Este mecanismo garante que qualquer novo servidor instalado com o Agent 2 via `zabbix_agente_linux.yml` (ou com o script PowerShell no Windows) seja cadastrado automaticamente no grupo correto sem intervenção manual no Zabbix.

3.2.3.4 Fase 4: Integração com Active Directory

Playbook `zabbix_ldap.yml`

Configura a autenticação LDAP/AD no Zabbix e cria os grupos de usuários com suas respectivas permissões.

```
1 ansible-playbook -i inventory/hosts.yml playbooks/zabbix_ldap.yml
```

O playbook executa as seguintes etapas:

1. **Copia o certificado da CA interna** (`certificados/capricornio-ca.crt`) para `/usr/local/share/ca-certificates/` no SPO-SRV-V01 e executa `update-ca-certificates` para validação TLS da conexão LDAPS
2. **Configura o TLS_REQCERT** em `/etc/ldap/ldap.conf` para `demand` (exige certificado válido)
3. **Cria 3 grupos de usuários** no Zabbix via API:
    - `Admins LDAP`: papel Super Admin, autenticação LDAP
    - `Users LDAP`: papel Admin, autenticação LDAP, permissão leitura/escrita em todos os grupos de host
    - `Viewers LDAP`: papel User, autenticação LDAP, permissão somente leitura em todos os grupos de host

4. **Cria o User Directory** “AD Capricornio” com os seguintes parâmetros:

Parâmetro	Valor
Host LDAP	ldaps://172.16.2.142:636
Base DN	`OU=Internos,OU=Usuarios,OU=Capricornio,DC=capricornio,DC=corp,DC=br`
Bind DN	`capricornio\zabbix`
Search attribute	`sAMAccountName`
JIT provisioning	habilitado

5. **Configura mapeamento de grupos AD para grupos Zabbix**:

Grupo AD	Grupo Zabbix	Papel resultante
`GG-Zabbix-Admin`	`Admins LDAP`	Super Admin
`GG-Zabbix-User`	`Users LDAP`	Admin
`GG-Zabbix-Viewer`	`Viewers LDAP`	User (somente leitura)

6. **Mantém o grupo** “Zabbix administrators” com autenticação interna para o usuário Admin local, garantindo acesso de emergência caso o LDAP fique indisponível

Playbook `grafana_ldap.yml`

Configura a autenticação LDAP/AD no Grafana usando o mesmo servidor AD.

```
ansible-playbook -i inventory/hosts.yml playbooks/grafana_ldap.yml
```

O playbook:

1. Copia `certificados/capricornio-ca.crt` para `/etc/grafana/ldap-ca.crt`
2. Copia o certificado para `/usr/local/share/ca-certificates/` e atualiza o store do sistema
3. Gera `/etc/grafana/ldap.toml` a partir do template `grafana-ldap.toml.j2`
4. Habilita LDAP em `/etc/grafana/grafana.ini` com `enabled = true` e `allow_sign_up = true`

O arquivo `ldap.toml` gerado configura:

- Servidor: `172.16.2.142`, porta 636, SSL habilitado, sem StartTLS (usa LDAPS puro)

- Certificado CA: `/etc/grafana/ldap-ca.crt`
- Search filter: `(sAMAccountName=%s)`
- Atributos mapeados: `givenName` (nome), `sn` (sobrenome), `sAMAccountName` (login), `mail` (e-mail)
- Mapeamento de grupos:

Grupo AD (DN completo)	Papel no Grafana
`CN=GG-Zabbix-Admin,OU=...`	Admin
`CN=GG-Zabbix-User,OU=...`	Editor
`CN=GG-Zabbix-Viewer,OU=...`	Viewer
(fallback, sem grupo)	Viewer

Playbook `zabbix_ad.yml`

Cadastra os 5 Domain Controllers da Capricórnio para monitoramento de disponibilidade dos serviços do AD. O monitoramento é feito por TCP simple check, sem necessidade de agente instalado nos DCs.

```
1 ansible-playbook -i inventory/hosts.yml playbooks/zabbix_ad.yml
```

Os 5 DCs monitorados:

Hostname	IP	Proxy
SPO-SRV-FAD01	172.17.2.140	(direto ao servidor)
SCL-SRV-V03	172.16.2.141	SCL-SRV-V08
SCL-SRV-V02	172.16.2.142	SCL-SRV-V08
BGP-SRV-VAD01	172.18.2.141	BGP-SRV-V08
BGP-SRV-V02	172.18.2.142	BGP-SRV-V08

Para cada DC, o playbook cria itens de verificação TCP nas seguintes portas:

Porta	Serviço AD	Trigger de falha
389	LDAP	Porta fechada por 5 minutos
636	LDAPS	Porta fechada por 5 minutos

Porta	Serviço AD	Trigger de falha
88	Kerberos KDC	Porta fechada por 5 minutos
3268	Global Catalog LDAP	Porta fechada por 5 minutos

A verificação por porta TCP é suficiente para detectar indisponibilidade dos serviços do AD sem precisar de agente nos DCs, que geralmente são servidores Windows com restrições de instalação de software de terceiros.

3.2.3.5 Fase 5: Monitoramento de Dispositivos de Rede

Playbook `zabbix_switches.yml`

Cadastra os 32 switches Cisco no Zabbix via API, distribuídos nos grupos por site, e associa cada switch ao proxy do respectivo site.

```
ansible-playbook -i inventory/hosts.yml playbooks/zabbix_switches.yml
```

Antes de executar, verificar que `automacao/vars/switches.yml` está com os IPs de gerência de cada switch preenchidos. Para gerar esse arquivo a partir da planilha de inventário:

```
cd scripts/
python3 gen_switch_vars.py
# Arquivo gerado: ../vars/switches_excel.yml
# Copiar ou mesclar com vars/switches.yml conforme necessario
```

O playbook cadastra cada switch com:

- **Interface SNMP**: tipo SNMPv3, porta 161/UDP, usando as credenciais globais de `automacao/vars/switches.yml` (usuário `zabbix_snmp`, autenticação SHA-256, privacidade AES-128, nível `authPriv`)
- **Template**: `Net Cisco IOS SNMP` (template oficial da comunidade Zabbix para switches Cisco com suporte a SNMP)
- **Grupo de host**: `Switches Cisco/BGP`, `/SCL` ou `/SPO` conforme o site do switch
- **Proxy**: `BGP-SRV-V08`, `SCL-SRV-V08` ou `SPO-SRV-V04` conforme o site

Ao final, o playbook exibe um resumo com o total de switches cadastrados por site. Para adicionar novos switches no futuro, basta adicionar entradas em `vars/switches.yml` e reexecutar o playbook; switches existentes não são duplicados (o módulo faz upsert por hostname).

Playbook `zabbix_criar_ups.yml`

Cadastra os UPS nos respectivos grupos de site via API Zabbix.

```
ansible-playbook -i inventory/hosts.yml playbooks/zabbix_criar_ups.yml
```

Os UPS são cadastrados com:

- **Interface SNMP**: SNMPv2c (todos os UPS usam SNMPv2c), porta 161/UDP, community definida em `vars/switches.yml`
- **Template**: `APC Smart-UPS by SNMP` para modelos APC; `PowerWalker UPS by SNMP` para modelos Phoenixtec
- **Grupo**: `UPS/BGP`, `UPS/SCL` ou `UPS/SPO` conforme o site

O playbook faz upsert por IP: se um host com o mesmo IP já existir, atualiza o hostname e os metadados sem criar duplicata.

3.2.3.6 Fase 6: Monitoramento de Servidores Linux

Playbook `zabbix_agente_linux.yml`

Instala e configura o Zabbix Agent 2 nos servidores Linux monitorados (fora da stack). O inventário é `automacao/inventory/linux_servidores.yml`.

```
ansible-playbook -i inventory/linux_servidores.yml playbooks/
    zabbix_agente_linux.yml
```

Para instalar em um servidor específico:

```
ansible-playbook -i inventory/linux_servidores.yml playbooks/
    zabbix_agente_linux.yml --limit=NOME-DO-HOST
```

O playbook determina automaticamente o proxy correto com base no grupo de inventário do servidor:

- Hosts no grupo `linux_bgp`: `ServerActive=BGP-SRV-V08`, `Server=BGP-SRV-V08`
- Hosts no grupo `linux_scl`: `ServerActive=SCL-SRV-V08`, `Server=SCL-SRV-V08`
- Hosts no grupo `linux_spo`: `ServerActive=SPO-SRV-V04`, `Server=SPO-SRV-V04`

Além da instalação do agente, o playbook:

1. Abre a porta 10050/TCP no UFW do servidor monitorado para aceitar conexões do proxy
2. Aguarda a porta 10050 ficar acessível (timeout configurável) antes de declarar sucesso
3. Configura o `HostMetadata` como `linux-server` para acionar o auto-registration no Zabbix

Após a execução, o host aparece automaticamente no grupo `Servidores Linux` com o template `Linux by Zabbix agent 2 active` aplicado, em até 5 minutos (intervalo padrão de auto-registration do Zabbix).

3.2.3.7 Fase 7: Monitoramento de Bancos de Dados

Playbook `configure_db_plugins.yml`

Instala os pacotes de plugin de banco de dados do Zabbix Agent 2 nos proxies e configura as sessões de conexão para cada instância monitorada. Este playbook é executado nos nós BGP-SRV-V08 e SCL-SRV-V08.

```
ansible-playbook -i inventory/hosts.yml playbooks/configure_db_plugins.
    yml
```

Tarefas executadas:

1. Instala os pacotes `zabbix-agent2-plugin-mssql` e `zabbix-agent2-plugin-postgresql` via APT
2. Habilita a diretiva `Include=/etc/zabbix/zabbix_agent2.d/plugins.d/*.conf` no `zabbix_agent2.conf`
3. Escreve as sessões de conexão de cada banco em `/etc/zabbix/zabbix_agent2.d/plugins.d/`:

Para SQL Server, gera entradas do tipo:

```
Plugins.MSSQL.Sessions.BGP_SRV_V04.Uri=sqlserver://172.18.2.12
Plugins.MSSQL.Sessions.BGP_SRV_V04.Username=zabbix
Plugins.MSSQL.Sessions.BGP_SRV_V04.Password=CHANGE_ME
```

Para PostgreSQL Veeam:

```
Plugins.PostgreSQL.Sessions.BGP_SRV_VAPP02.Uri=postgresql://
    172.18.2.5:5432
Plugins.PostgreSQL.Sessions.BGP_SRV_VAPP02.Username=veeam.bkp
Plugins.PostgreSQL.Sessions.BGP_SRV_VAPP02.Password=CHANGE_ME
```

4. Valida a sintaxe do `zabbix_agent2.conf` com `zabbix_agent2 -T`
5. Reinicia o serviço `zabbix-agent2`
6. Testa a conectividade com `zabbix_get` antes de declarar sucesso:

```
zabbix_get -s 127.0.0.1 -p 10050 -k "mssql.ping[BGP_SRV_V04]"
zabbix_get -s 127.0.0.1 -p 10050 -k "pgsql.ping[BGP_SRV_VAPP02]"
```

Playbook `zabbix_databases.yml`

Cadastra as 7 instâncias de banco de dados no Zabbix via API e configura as macros de conexão em cada host.

```
ansible-playbook -i inventory/hosts.yml playbooks/zabbix_databases.yml
```

As instâncias cadastradas e suas macros de host:

SQL Server (3 instâncias): cada host recebe o template `MSSQL by Zabbix agent 2` e as macros `{$MSSQL.INSTANCE}` (nome da instância SQL), `{$MSSQL.USERNAME}` e `{$MSSQL.PASSWORD}` com as credenciais do usuário de monitoramento. A coleta é feita pelo plugin MSSQL do Zabbix Agent 2 rodando no proxy do site.

PostgreSQL Veeam (BGP-SRV-VAPP02): template `PostgreSQL by Zabbix agent 2`, macros `{$PG.CONNSTRING}` com a string de conexão e `{$PG.USER}` / `{$PG.PASSWORD}` com as credenciais do usuário de leitura.

Oracle Systextil ERP: cadastrado como novo host `BGP-SYS-ORACLE01` (IP 10.20.1.100) no grupo `Bancos de Dados`, proxy `BGP-SRV-V08`. Template `Oracle by Zabbix agent 2` com macros de connection string, usuário e serviço Oracle.

ClickHouse (SCL-SRV-V05, porta 8123): monitorado via HTTP Agent (não requer plugin instalado no agente). O playbook cria 6 itens individuais via HTTP para as métricas: uptime, consultas em execução, total de consultas, conexões TCP, uso de memória e contagem de tabelas. Macros `{$CLICKHOUSE.HOST}`, `{$CLICKHOUSE.PORT}`, `{$CLICKHOUSE.USER}` e `{$CLICKHOUSE.PASSWORD}`.

Playbook `diagnostico_db_plugins.yml`

Verifica a conectividade de cada plugin de banco de dados após a configuração. Executar sempre que houver mudança de senha ou problema de coleta.

```
ansible-playbook -i inventory/hosts.yml playbooks/
    diagnostico_db_plugins.yml
```

Executa em sequência:

1. **Teste de porta TCP** via `netcat -zv` para cada IP/porta de banco de dados
2. **Teste de ping do plugin** via `zabbix_get` para `mssql.ping[*]` e `pgsql.ping[*]`
3. **Análise de log**: exibe as últimas 50 linhas do `/var/log/zabbix/zabbix_agent2.log` filtradas por palavras-chave de erro e conectividade

O resultado esperado para cada banco é `1` (sucesso) no teste de ping do plugin.

3.2.3.8 Fase 8: Alertas e Notificações

O Zabbix foi configurado com dois canais de notificação para envio proativo de alertas: Telegram e Microsoft Teams. Ambos os canais recebem notificações de abertura, atualização e encerramento de problemas, com filtro por severidade para evitar ruído desnecessário.

Media Type: Telegram

O Media Type do Telegram foi configurado em Administration > Media Types utilizando o tipo “Webhook” nativo do Zabbix 7.0. A integração usa um bot dedicado criado via BotFather, com o token de autenticação e o ID do chat/grupo de destino configurados como parâmetros do webhook. O script JavaScript embutido no Media Type formata a mensagem com emoji de severidade, nome do host, descrição do problema e horário de ocorrência, e envia via HTTP POST para a API do Telegram (`api.telegram.org/bot{TOKEN}/sendMessage`).

O Telegram é utilizado como canal de notificação imediata para a equipe de plantão, pela natureza assíncrona e pela disponibilidade em dispositivos móveis.

Media Type: Microsoft Teams

O Media Type do Teams foi configurado via webhook de entrada (Incoming Webhook) criado em um canal dedicado (“Alertas Zabbix”) no Teams da Capricórnio. O webhook URL é configurado como parâmetro do Media Type no Zabbix. As mensagens são formatadas como Adaptive Cards com campos estruturados: severidade, host, grupo de host, descrição do problema, IP e link direto para o evento no Zabbix.

O Teams serve como canal de registro formal e visibilidade da equipe de TI durante o horário comercial.

Actions de Notificação

Duas Actions foram configuradas em Configuration > Actions > Trigger Actions:

A Action “Notificar Telegram” envia alertas para o canal Telegram nos seguintes eventos, com os filtros de severidade:

Evento	Severidades notificadas
Problema detectado	Warning, Average, High, Disaster
Problema atualizado	High, Disaster
Problema recuperado	Warning, Average, High, Disaster

A Action “Notificar Teams” segue a mesma lógica com envio para o webhook do canal Teams. O usuário Zabbix associado a cada Media Type tem e-mail e parâmetros de mídia configurados em Administration > Users > Media.

A macro `{$ALERT.EMAIL}` já definida globalmente como `infraestrutura.n2@capricornio.com.br` está disponível para eventual configuração de um terceiro canal por e-mail, sem necessidade de alterar as Actions existentes.

3.2.3.9 Fase 10: Dashboards e Monitoramento Web

Playbook `grafana_dashboards.yml`

Importa os 9 dashboards JSON para o Grafana e configura o provisioning automático.

```
ansible-playbook -i inventory/hosts.yml playbooks/grafana_dashboards.
    yml
```

O playbook:

1. Cria a estrutura de diretórios em `/var/lib/grafana/dashboards/` com subdiretórios por categoria: `00-overview`, `rede`, `servidores`, `infra-dados`, `energia`
2. Copia os arquivos JSON dos dashboards de `dashboards-grafana/` para os subdiretórios correspondentes
3. Copia o arquivo de provisioning `dashboards-grafana/provisioning/dashboards.yaml` para `/etc/grafana/provisioning/dashboards/`
4. Reinicia o Grafana para carregar os dashboards do diretório de provisioning

O provisioning automático garante que qualquer alteração nos arquivos JSON (correção de painel, novo indicador) seja aplicada simplesmente copiando o arquivo atualizado e reiniciando o Grafana, sem importação manual pela interface.

Após a importação, os dashboards ficam disponíveis em `http://172.17.2.53:3000/dashboards`.

Playbook `zabbix_web_monitor.yml`

Cria cenários de monitoramento web (HTTP) e verificações de certificados SSL no Zabbix.

```
ansible-playbook -i inventory/hosts.yml playbooks/zabbix_web_monitor.
    yml
```

Todos os cenários são agrupados sob o host `CAPRICORNIO-WEB` (172.17.2.51) com monitoramento direto pelo servidor (sem proxy). O playbook cria 22 cenários de URL com verificação de código HTTP e 8 verificações de certificado SSL:

Cenários web (HTTP 200 esperado):

URL monitorada	Aplicação
SinergyRH Ponto e SinergyRH Folha	Sistema de ponto e folha de pagamento
Plusoft OMNI (tenant Capricornio)	CRM omnichannel
Picanol PicConnect	Sistema de automação industrial

URL monitorada	Aplicação
Paytrack	Gestão de despesas
NDD Print 360	Gestão de impressão
Lansweeper	Inventário de ativos de TI
HAProxy Systextil ERP	Balanceador do ERP Systextil
Deps API, Deps Vivo, Deps Home, Deps Cross	APIs de comunicação corporativa
Capricornio Site	Site corporativo
Base-B	Sistema interno
CLP 1, CLP 2, CLP 3	Controladores lógicos programáveis industriais
Projuris	Sistema jurídico
Node-RED instâncias 1, 2 e 3 (portas 1880/1881/1882)	Automação de processos
Cisco Dashboard	Dashboard de gerência Cisco

Certificados SSL (verificação de validade):

Domínio	Trigger de aviso	Trigger crítica
GLPI ServiceDesk	30 dias antes do vencimento	7 dias antes
OpManager	30 dias	7 dias
Lansweeper	30 dias	7 dias
Dimep	30 dias	7 dias
DepsNet	30 dias	7 dias
Deps API	30 dias	7 dias
Cisco Dashboard	30 dias	7 dias
Capricornio Site	30 dias	7 dias

3.2.3.10 Fase 11: Self-Monitoring dos Proxies

Playbook `configure_proxy_monitoring.yml`

Configura o Agent 2 instalado nos proxies para que cada proxy monitore a si mesmo, permitindo que o Zabbix Server saiba sobre a saúde do agente em cada nó de proxy.

```
ansible-playbook -i inventory/hosts.yml playbooks/
    configure_proxy_monitoring.yml
```

O playbook atualiza as diretivas `Server=` e `ServerActive=` no `zabbix_agent2.conf` de cada proxy para incluir `127.0.0.1` (acesso local), o IP do Zabbix Server e o IP do próprio proxy. Isso permite:

- Coleta de métricas do host pelo Zabbix Server diretamente
- Auto-monitoramento local via `zabbix_get` de diagnóstico

Após a execução, verifica com `zabbix_get` local:

```
zabbix_get -s 127.0.0.1 -p 10050 -k "agent.ping"      # deve retornar 1
zabbix_get -s 127.0.0.1 -p 10050 -k "agent.hostname"   # deve retornar
    o hostname do proxy
```

3.2.3.11 Playbooks de Manutenção e Operação

Playbook `zabbix_reset_discovery.yml`

Força a reexecução imediata das regras de descoberta de rede sem aguardar o intervalo de 1 hora. Útil após adicionar dispositivos à rede ou quando um host sumiu do Zabbix e precisa ser redescoberto.

```
ansible-playbook -i inventory/hosts.yml playbooks/
    zabbix_reset_discovery.yml
```

O playbook:

1. Deleta as 3 regras de discovery existentes via API
2. Recria as regras com delay inicial de `1m` (1 minuto) em vez de `1h`
3. Aguarda 90 segundos para que os proxies executem a varredura
4. Atualiza as regras para retornar ao delay de `1h`

Playbook `zabbix_atribuir_proxies.yml`

Reatribui automaticamente os proxies de todos os hosts com base no segundo octeto do IP da interface principal. Útil após migração de hosts entre sites ou após importação de hosts com proxy incorreto.

```
ansible-playbook -i inventory/hosts.yml playbooks/
    zabbix_atribuir_proxies.yml
```

Lógica de atribuição:

- IP no range 172.**16**.x.x (segundo octeto = 16): atribui proxy `SCL-SRV-V08`
- IP no range 172.**17**.x.x (segundo octeto = 17): atribui proxy `SPO-SRV-V04`
- IP no range 172.**18**.x.x (segundo octeto = 18): atribui proxy `BGP-SRV-V08`

- IP `127.0.0.1` (Zabbix Server self-monitoring): ignorado

Playbook `zabbix_classificar_hosts.yml`

Classifica hosts que estão no grupo "Em Classificação", movendo-os para o grupo correto e aplicando o template adequado. Usa três fontes de dados em ordem de prioridade:

```
ansible-playbook -i inventory/hosts.yml playbooks/
    zabbix_classificar_hosts.yml
```

1. **Planilha de inventário** (`docs/Sws_Capricornio.xlsx`): se o IP do host constar na planilha, usa o nome, tipo, site e template definidos nela
2. **DNS reverso (PTR)**: se o hostname seguir o padrão `SITE-TIPO99` (ex: `BGP-SW-01`), extrai o site do prefixo (16=SCL, 17=SPO, 18=BGP) e o tipo do segmento do meio
3. **Fallback**: host permanece em "Em Classificação" se não houver informação suficiente para classificar

Playbook `zabbix_criar_hosts_descobertos.yml`

Cria no Zabbix os hosts descobertos pelas regras de discovery que ainda não foram adicionados. Consulta os `dhosts` (discovered hosts) via API e compara com os hosts já cadastrados:

```
ansible-playbook -i inventory/hosts.yml playbooks/
    zabbix_criar_hosts_descobertos.yml
```

1. Consulta todos os `dhosts` retornados pelas regras de discovery
2. Extrai os IPs únicos das interfaces descobertas
3. Compara com a lista de IPs já cadastrados como hosts no Zabbix
4. Para cada IP novo, cria o host no grupo "Em Classificação" com interface SNMPv3
5. Exibe um resumo: IPs descobertos, já monitorados, e novos criados

Após executar este playbook, rodar `zabbix_classificar_hosts.yml` para mover os hosts recém-criados para os grupos corretos.

Playbook `zabbix_atualizar_snmp_cisco.yml`

Atualiza as credenciais SNMPv3 de todos os hosts no grupo "Switches Cisco/*" para as credenciais definidas em `vars/switches.yml`. Executar quando as senhas SNMPv3 forem trocadas nos switches.

```
ansible-playbook -i inventory/hosts.yml playbooks/
    zabbix_atualizar_snmp_cisco.yml
```

O playbook busca todos os hosts nos grupos filhos de "Switches Cisco" e atualiza a interface SNMP de cada um para SNMPv3 com `authprotocol=SHA256` e `privprotocol=AES128`.

Playbook `zabbix_fix_fortigate_snmp.yml`

Corrige inconsistências na configuração dos hosts FortiGate. Executar se houver problemas de coleta em firewalls FortiGate.

```
ansible-playbook -i inventory/hosts.yml playbooks/
    zabbix_fix_fortigate_snmp.yml
```

Aplica quatro correções independentes:

- **Fix A**: hosts FortiGate com template HTTP mas sem interface SNMP, migrados para template SNMP
- **Fix B**: hosts FortiGate sem template algum, com adição do template correto
- **Fix C**: interfaces SNMP existentes com versão errada (v1 ou v2c), migradas para SNMPv3 com credenciais atualizadas
- **Fix D**: hosts FortiGate sem interface SNMP cadastrada, com criação da interface SNMPv3

Playbook `zabbix_fix_qnap_hosts.yml`

Corrige erros nos item prototypes do template QNAP relacionados a tipos de valor incompatíveis com as respostas SNMP reais dos modelos implantados.

```
ansible-playbook -i inventory/hosts.yml playbooks/zabbix_fix_qnap_hosts
    .yml
```

Correções aplicadas no template (propagam para todos os hosts QNAP):

- Altera o `value_type` dos itens de Status e Temperatura de `Numeric` para `Text` (esses OIDs retornam strings, não números)
- Altera o `value_type` dos itens de Velocidade e Status de FAN de `Numeric` para `Text`
- Desabilita os itens de "Nome do FAN" pois o OID correspondente não existe nos modelos QNAP instalados

Playbook `zabbix_fix_itens_template.yml`

Corrige erros similares em outros templates instalados.

```
ansible-playbook -i inventory/hosts.yml playbooks/
    zabbix_fix_itens_template.yml
```

Correções aplicadas:

- **Template Aruba IAP**: desabilita 6 itens de rádio (ex: potência de transmissão, número de clientes por banda) cujos OIDs não existem nos modelos de AP instalados; evita erros constantes de `SNMP agent item unsupported`
- **Template SNMP QNAP**: aplica as mesmas correções de `value_type` e itens FAN do playbook anterior (útil quando o template foi reimportado)

- **Template Zabbix Server**: desabilita o item `haservice` que verifica o serviço de HA do Zabbix; como o ambiente não usa HA de servidor Zabbix, esse item gera erro permanente de `item not supported`

3.2.4 Mecanismos de Descoberta do Zabbix

O Zabbix emprega dois mecanismos complementares de descoberta automática: Network Discovery (para dispositivos sem agente, monitorados via SNMP ou ICMP) e Auto-registration (para hosts com o Zabbix Agent 2 instalado).

3.2.4.1 Descoberta de Rede (Network Discovery)

O Network Discovery executa varreduras periódicas de sub-redes predefinidas usando verificações configuráveis. Na implantação da Capricórnio, cada site tem sua própria regra varrendo a sub-rede de gerência:

Como funciona o ciclo de discovery:

1. O proxy do site inicia a varredura da sub-rede a cada 3600 segundos
2. Para cada IP no range, o proxy tenta: (a) ICMP ping para verificar alcançabilidade, e (b) leitura SNMP do OID `.1.3.6.1.2.1.1.1.0` (sysDescr) com as credenciais SNMPv3 globais
3. Os resultados são enviados ao Zabbix Server e armazenados na tabela `dservices` do banco
4. O Zabbix Server avalia as Discovery Actions configuradas
5. Para cada IP que "apareceu" (passou de não-encontrado para encontrado), a action correspondente é executada
6. A action examina o valor do sysDescr retornado e adiciona o host ao grupo correspondente

O que acontece com um dispositivo novo:

Quando um switch novo for adicionado à rede de gerência (sub-rede .23. do site), sem nenhuma ação manual:

1. Na próxima varredura (em até 1 hora), o proxy detecta o novo IP
2. O sysDescr identifica o dispositivo como CBS350 (ou similar)
3. A action cria o host no Zabbix no grupo "Switches Cisco/SITE", aplica o template SNMP do Cisco e associa ao proxy do site
4. O Zabbix começa a coletar dados automaticamente

Para forçar a descoberta imediata sem aguardar 1 hora, usar o playbook `zabbix_reset_discovery.yml`.

Visualizar hosts descobertos:

No Zabbix: Monitoring > Discovery. A tela mostra os hosts descobertos por regra, com seus serviços (portas/OIDs respondendo) e quando foram vistos pela última vez.

Visualizar a fila de classificação:

Hosts descobertos mas ainda sem classificação ficam em: Monitoring > Hosts, filtro por grupo "Em Classificação".

3.2.4.2 Registro Automático de Agentes (Auto-registration)

O Auto-registration é o mecanismo para servidores (Windows, Linux, Hyper-V) que têm o Zabbix Agent 2 instalado. Ao iniciar pela primeira vez, o agente envia um pacote de registro ao `ServerActive` configurado, incluindo seu hostname, IP e o valor do campo `HostMetadata`.

Como funciona o ciclo de auto-registration:

1. O Agent 2 inicia e envia requisição de auto-registro para o ServerActive (proxy ou servidor)
2. O Zabbix Server avalia as Auto-registration Actions configuradas em `zabbix_infra_setup.yml`
3. A action filtra pelo valor do `HostMetadata` enviado pelo agente
4. O host é criado no Zabbix com o hostname, grupo e template corretos

HostMetadata configurados:

O valor de `HostMetadata` é definido no `zabbix_agent2.conf` durante a instalação pelo Ansible. Os playbooks `zabbix_agente_linux.yml` (Linux) e o script PowerShell para Windows configuram os seguintes valores:

- `linux-server`: servidores Linux Ubuntu, qualquer site
- `windows-server`: servidores Windows Server 2022 e workstations
- `hyperv`: hosts Hyper-V bare metal

Como registrar um novo servidor Windows manualmente:

1. Fazer o download do Zabbix Agent 2 para Windows no site oficial ou usar o pacote disponibilizado pela 4Linux
2. Instalar com o script PowerShell disponível em `automacao/scripts/hyperv-deploy/` (ou com instalador MSI)
3. Configurar o `zabbix_agent2.conf` no Windows com:

```
Server=IP_DO_PROXY_DO_SITE
ServerActive=IP_DO_PROXY_DO_SITE
Hostname=NOME-DO-SERVIDOR
HostMetadata=windows-server
```

4. Iniciar o serviço `Zabbix Agent 2`
5. Em até 5 minutos, o host aparece automaticamente em “Servidores Windows/SITE” no Zabbix

3.2.5 Templates Utilizados

Os templates abaixo foram utilizados na implantação. Todos são templates oficiais distribuídos com o Zabbix 7.0 ou templates da comunidade Zabbix, com modificações pontuais aplicadas pelos playbooks de correção.

Template	Tipo de dispositivo	Método de coleta	Principais métricas monitoradas
Net Cisco IOS SNMP	Switches Cisco CBS350/Catalyst	SNMPv3	CPU, memória, uptime, status de interfaces, utilização de banda, erros TX/RX, status PoE, status de uplinks, temperaturas
FortiGate by HTTP	Firewalls FortiGate	API REST HTTPS	CPU, memória, sessões ativas, throughput, status de interfaces, status de VPN IPSec/SSL, tabela de roteamento, licenças
QNAP NAS by SNMP	NAS QNAP	SNMPv3	Status e uso de volumes, temperatura de discos físicos, status RAID, uso de CPU e memória, tráfego de rede
APC Smart-UPS by SNMP	UPS APC	SNMPv2c	Autonomia de bateria, percentual de carga, tensão de entrada/saída, temperatura interna, status da bateria

Template	Tipo de dispositivo	Método de coleta	Principais métricas monitoradas
PowerWalker UPS by SNMP	UPS Phoenixtec	SNMPv2c	Autonomia, carga, tensão de entrada/saída, temperatura, status geral
Aruba IAP by SNMPv3	APs Aruba + controllers	SNMPv3	Clientes associados por AP, throughput por rádio, status de uplink, status do controller, canal e potência de transmissão
Windows OS by Zabbix agent active	Servidores Windows 2022	Agent 2 ativo	CPU (por núcleo), memória, disco (por volume), rede (por interface), serviços, processos, atualizações pendentes, logs de evento
Linux by Zabbix agent 2 active	Servidores Linux Ubuntu	Agent 2 ativo	CPU (load, idle, iowait), memória (free, cached, buffers), disco (uso, iops, latência), rede (bytes/s, erros), processos, swap
Hyper-V by Zabbix agent 2	Hosts Hyper-V	Agent 2 ativo	Status de VMs (running/stopped), CPU do host, memória do host, I/O de disco, VMs por estado

Template	Tipo de dispositivo	Método de coleta	Principais métricas monitoradas
Dell iDRAC by SNMP	Servidores Dell PowerEdge	SNMPv3	Status de PSU, temperatura de CPU e chassi, velocidade de ventiladores, status de discos físicos e virtuais (RAID), status de bateria do RAID controller
MSSQL by Zabbix agent 2	SQL Server 2022	Agent 2 (plugin MSSQL)	Conexões ativas, queries/s, deadlocks, tamanho de bancos, status de AG (Always On), tamanho de log, wait statistics
PostgreSQL by Zabbix agent 2	PostgreSQL 16	Agent 2 (plugin PostgreSQL)	Conexões ativas vs. máximo, queries lentas, tamanho de bancos, tabelas inchadas (bloat), WAL gerado/s, cache hit ratio
MySQL by Zabbix agent 2	MySQL (Lansweeper)	Agent 2 (plugin MySQL)	Conexões, queries/s, innodb buffer pool, slow queries, replicação (se houver)
ClickHouse HTTP items	ClickHouse	HTTP Agent	Uptime, queries em execução, queries totais/s, conexões TCP, uso de memória, contagem de tabelas

Template	Tipo de dispositivo	Método de coleta	Principais métricas monitoradas
Oracle by Zabbix agent 2	Oracle (Systextil ERP)	Agent 2 (plugin Oracle)	Sessões ativas, tablespace usage, redo log rate, SGA/PGA usage, wait events, locks

3.2.6 Macros Globais de Limiares

As macros abaixo controlam os limiares de triggers para todos os hosts. Para ajustar um limiar globalmente, alterar o valor da macro em Administration > Macros e todas as triggers dos templates que a referenciam serão ajustadas automaticamente.

Macro	Valor padrão	Tipo de trigger
{$CPU.UTIL.CRIT}	90%	Crítica: CPU acima do limiar por 5 minutos
{$MEMORY.UTIL.MAX}	90%	Aviso: memória usada acima do limiar
{$IF.UTIL.MAX}	80%	Aviso: interface de rede saturada
{$DISK.PFREE.MIN.WARN}	20%	Aviso: disco com menos de 20% livre
{$DISK.PFREE.MIN.CRIT}	10%	Crítica: disco com menos de 10% livre
{$UPS.BATTERY.CAPACITY.MIN.WARN}	40%	Aviso: bateria de UPS abaixo de 40%
{$CERT.EXPIRY.WARN}	30 dias	Aviso: certificado SSL vence em menos de 30 dias

Para ajustar um limiar apenas para um host específico (sem afetar os demais), adicionar a macro com o mesmo nome diretamente no host em Configuration > Hosts > Macros. A macro do host tem precedência sobre a macro global.

3.2.7 Scripts de Automação e Diagnóstico

3.2.7.1 `zabbix_audit_hosts.py`

Script Python de auditoria completa da base de hosts do Zabbix. Gera um relatório CSV com o estado de cada host, identificando problemas de configuração e conectividade.

```
cd automacao/scripts/
python3 zabbix_audit_hosts.py
# Saida: docs/zabbix_diagnostico_AAAAMMDD_HHMMSS.csv
```

O script se conecta à API do Zabbix, consulta todos os hosts e classifica cada um em uma das seguintes categorias de status:

Status	Significado
OK	Host coletando normalmente, sem erros de item
OK (device-side)	Host alcançável, com erros menores no dispositivo monitorado
COM ERROS	Host alcançável mas com itens com erro de coleta
SEM CONEXÃO	Host não está respondendo (agente ou SNMP inacessível)
PENDENTE CONFIG	Host sem template ou sem interface configurada

O relatório CSV inclui: IP, hostname, visible name, site, tipo de dispositivo, grupos, templates, versão SNMP, status de disponibilidade (Agent/SNMP/ICMP/HTTP), contagem de erros de item, detalhes dos primeiros 5 erros e sugestões de ação corretiva.

Para executar, é necessário configurar as credenciais da API Zabbix no início do script (URL, usuário, senha) ou via variáveis de ambiente.

3.2.7.2 `gen_switch_vars.py`

Gera o arquivo de variáveis de switches (`vars/switches_excel.yml`) a partir da planilha de inventário em Excel.

```
cd automacao/scripts/
python3 gen_switch_vars.py
# Le: docs/Sws_Capricornio.xlsx
# Gera: vars/switches_excel.yml
```

O script lê a planilha a partir da linha 3 (linha 1 é cabeçalho, linha 2 é subtítulo), extrai as colunas Nome, IP, Modelo e Localização, e determina o site a partir do segundo octeto do IP. O arquivo gerado é um dicionário YAML indexado por IP:

```
"172.18.23.10":
  name: "BGP-SW-01"
  model: "CBS350-48P"
  location: "Rack A - BGP"
  site: "BGP"
```

Requer a biblioteca `openpyxl`: `pip3 install openpyxl`.

3.2.7.3 zabbix_classificar_planilha.py

Classifica e migra hosts do Zabbix com base na planilha exportada do OpManager. Útil para migrações iniciais e para reclassificar lotes de hosts importados.

```
cd automacao/scripts/
# Modo simulacao (ver o que seria feito, sem executar)
python3 zabbix_classificar_planilha.py --dry-run

# Executar a classificacao real
python3 zabbix_classificar_planilha.py
# Le: docs/relacao_opmanager.xlsx (aba Planilha1)
```

O script mapeia 44 tipos de dispositivo definidos na planilha do OpManager para grupos, templates e tipos de interface no Zabbix. Para cada host na planilha, o script:

1. Localiza o host no Zabbix pelo IP
2. Compara o hostname atual com o nome na planilha; atualiza se diferente
3. Move para o grupo correto se ainda estiver em “Em Classificação”
4. Aplica o template correspondente ao tipo de dispositivo
5. Exibe estatística final: hosts já corretos, atualizados, criados e ignorados

Para hosts com nome duplicado (como Virtual Controllers Aruba que podem ter o mesmo nome), adiciona o sufixo `-VC` automaticamente para evitar colisão.

3.2.7.4 zabbix_server_tune.sh

Script de ajuste de performance do Zabbix Server. Executar diretamente no SPO-SRV-V01 como root ou via sudo.

```
# No SPO-SRV-V01
sudo bash /opt/scripts/zabbix_server_tune.sh
```

O script cria um backup de `/etc/zabbix/zabbix_server.conf` com timestamp antes de qualquer modificação (`zabbix_server.conf.bak.AAAAMMDD_HHMMSS`), aplica os ajustes de `ValueCacheSize=64M` e `StartPingers=3`, reinicia o serviço e aguarda 10 segundos antes de verificar se o serviço voltou ao status `active (running)`.

3.2.7.5 Scripts de Roteamento VPN

Os scripts em `automacao/scripts/` gerenciam as rotas necessárias quando a equipe da 4Linux acessa os servidores da Capricórnio via VPN PPP. Um problema comum é que o Docker (instalado no control node) cria rotas /16 que têm precedência sobre as rotas mais específicas da VPN.

`vpn-routes-up.sh`: adiciona rotas /32 para os 6 servidores da stack via interface PPP. Executar manualmente após conectar à VPN:

```
bash scripts/vpn-routes-up.sh ppp0
```

`ppp-ip-up.sh` e `ppp-ip-down.sh`: hooks do pppd para adicionar/remover as rotas automaticamente quando a VPN sobe/desce. Instalar em `/etc/ppp/ip-up.d/` e `/etc/ppp/ip-down.d/` no control node para operação automática:

```
sudo cp scripts/ppp-ip-up.sh /etc/ppp/ip-up.d/capricornio
sudo cp scripts/ppp-ip-down.sh /etc/ppp/ip-down.d/capricornio
sudo chmod +x /etc/ppp/ip-up.d/capricornio /etc/ppp/ip-down.d/
    capricornio
```

3.3 Descrição do Ambiente Entregue

Ao encerramento do atendimento, a stack de observabilidade da Capricórnio encontra-se operacional nos três sites com monitoramento ativo de 250+ ativos.

Item	Situação Anterior	Situação Atual	Status
Solução de monitoramento	OpManager com cobertura parcial	Zabbix 7.0 LTS + Grafana 13	Implantado
Banco de dados de monitoramento	Embutido no OpManager	PostgreSQL 16 dedicado (SPO-SRV-V02, 172.17.2.52)	Implantado
Cobertura de sites	Parcial	BGP, SCL e SPO com proxy dedicado por site	Implantado

Item	Situação Anterior	Situação Atual	Status
Switches Cisco monitorados	Parcial	32 switches via SNMPv3 (authPriv SHA-256/AES-128)	Implantado
Firewalls FortiGate	Parcial	5 FortiGates via API REST HTTPS	Implantado
UPS monitorados	Não monitorados	27 UPS Phoenixtec e APC via SNMP	Implantado
NAS QNAP monitorados	Parcial	7 NAS via SNMPv3	Implantado
APs Aruba monitorados	Não monitorados	113 APs + 6 controllers via SNMPv3	Implantado
Servidores Windows	Parcial	31 servidores + 4 workstations via Agent 2	Implantado
Servidores Linux	Parcial	8 servidores via Agent 2	Implantado
Hosts Hyper-V	Não monitorados	6 hosts via Agent 2	Implantado
iDRAC Dell	Não monitorados	7 servidores via SNMPv3	Implantado
Bancos de dados	Não monitorados	7 instâncias (SQL Server, PostgreSQL, MySQL, ClickHouse, Oracle)	Implantado
Domain Controllers AD	Não monitorados	5 DCs com verificação de 4 portas cada	Implantado
Endpoints web monitorados	Não monitorados	22 cenários HTTP	Implantado
Certificados SSL	Não monitorados	8 domínios com alerta antecipado	Implantado
Integração AD/LDAP	Não integrado	LDAPS com 3 grupos de acesso no Zabbix e Grafana	Implantado

Item	Situação Anterior	Situação Atual	Status
Dashboards executivos	Ausentes	9 dashboards por categoria no Grafana 13	Implantado
Hardening dos servidores	Ausente	SSH Ed25519, UFW, 25 params sysctl, unattended-upgrades em 6 nós	Implantado
Automação IaC	Ausente	26 playbooks Ansible versionados em repositório Git	Entregue
Documentação técnica	Ausente	Manual de infraestrutura, guias de operação, inventário completo	Entregue

4 Problemas e Riscos Identificados

Severidade	Observação	Impacto Potencial	Sugestão
Média	PostgreSQL em instância única sem replicação	Falha no SPO-SRV-V02 exige restore manual antes de retomar coleta	Considerar replicação streaming com Patroni ou rotina de backup diário com `pg_basebackup` em próxima fase
Média	Acesso ao Grafana e ao frontend Zabbix via HTTP	Credenciais trafegam sem criptografia na rede interna	Os certificados da CA interna já estão no repositório; habilitar TLS no Nginx é o próximo passo natural
Baixa	Credenciais armazenadas em arquivos YAML não cifrados no repositório	Risco de exposição caso o repositório seja compartilhado indevidamente	Migrar para Ansible Vault mantém o fluxo de automação sem alterações nos playbooks

5 Conclusões e Recomendações

A implantação da stack de observabilidade foi concluída dentro do prazo e com escopo integral entregue. A Capricórnio dispõe agora de uma plataforma de monitoramento moderna, distribuída em três sites, cobrindo 250+ ativos com visibilidade unificada: switches, firewalls, APs, NAS, UPS, servidores Windows e Linux, hosts Hyper-V, iDRAC Dell, sete instâncias de bancos de dados, 22 endpoints web, oito certificados SSL e cinco Domain Controllers do Active Directory. O acesso é integrado ao AD corporativo, os dashboards Grafana entregam visibilidade executiva por categoria de ativo, e toda a configuração está codificada em 26 playbooks Ansible, versionados e prontos para reutilização.

A seguir, algumas oportunidades de evolução da solução que a equipe de TI da Capricórnio pode considerar para fortalecer ainda mais a plataforma entregue.

Evolução 1: Alta disponibilidade do PostgreSQL. O banco de dados foi implantado em instância dedicada com excelente separação de recursos. Para elevar a resiliência da solução a um próximo nível, recomenda-se avaliar a implementação de replicação streaming com Patroni ou a configuração de backups automáticos diários com `pg_basebackup`. Um procedimento de restore testado e documentado garante que, em um cenário de falha de hardware, a retomada seja rápida e previsível.

Evolução 2: TLS nos frontends Grafana e Zabbix. Os certificados da CA interna da Capricórnio já estão no repositório do projeto (`certificados/capricornio-ca.crt`), o que torna esta evolução simples de executar. Habilitar HTTPS no Nginx de SPO-SRV-V01 e SPO-SRV-V03 é um passo natural de maturidade para o ambiente, e a URL `https://172.17.2.53` já está prevista na documentação do projeto.

Evolução 3: Gestão de segredos com Ansible Vault. À medida que a equipe passa a executar os playbooks de forma autônoma, cifrar as variáveis sensíveis com Ansible Vault é uma boa prática que protege credenciais sem alterar nenhum playbook existente. A chave do vault é armazenada fora do repositório e fornecida no momento da execução, um fluxo simples e compatível com pipelines CI/CD.

Evolução 4: Revisão periódica do inventário. O script `automacao/scripts/zabbix_audit_hosts.py`, entregue como parte do projeto, gera em menos de 5 minutos um relatório CSV com o estado de cada host: conectividade, erros de coleta e sugestões de correção. Executá-lo trimestralmente permite manter a base de monitoramento limpa e precisa à medida que a infraestrutura evolui, garantindo que novos ativos entrem no escopo e equipamentos descomissionados sejam retirados.